ChatGPT

Executive Summary

GridCast AI is a student-developed predictive analytics system for **short-term electricity demand forecasting**. The goal is to produce accurate hourly demand predictions (e.g. 24 hours ahead) using machine learning, and to serve them via a web application and API. This project will demonstrate a full ML lifecycle: data collection, feature engineering, model training (with multiple baselines and state-of-the-art algorithms), rigorous evaluation, and deployment to a Streamlit dashboard with a FastAPI backend. Success will be measured by forecast accuracy (MAE/RMSE improvements over naive benchmarks), responsive APIs (<500 ms latency), and system reliability. The deliverables include code, trained models, documentation (README, PRD), and a demo. All tools will use free or student-eligible tiers (Python stack, GitHub, Streamlit, free weather APIs, etc.). A detailed timeline (see *Appendix*) guides the work.

1. Project Overview and Goals

- **Problem:** Electricity grid operators and energy planners need reliable short-term forecasts of electricity demand (load) to balance supply, schedule generation, and prevent outages. Errors can lead to blackouts or wasted resources.
- **Objective:** Build a predictive analytics platform that forecasts next-24-hour electricity demand at hourly granularity. It will ingest historical load (plus exogenous data), train ML models, and serve forecasts via an interactive dashboard and API.
- **Scope:** Use open datasets (e.g. national/regional load histories, weather), standard ML tools, and deploy on free cloud services. All work must be reproducible and suitable for a student project.
- **Key Goals:**
- **Accuracy:** Achieve substantially better forecasts than simple baselines (e.g. 20–30% lower MAE).
- **Robustness:** Use proper time-series validation (chronological splits), hyperparameter tuning, and model selection.
- **Explainability:** Provide feature-importance (e.g. SHAP) to justify forecasts.
- **Interactivity:** Develop a Streamlit UI with plots of actual vs predicted, error metrics, and user queries for custom forecasts.
- **Deployment:** Host the application (Streamlit Cloud or similar) and a FastAPI endpoint for programmatic predictions.
- **Reproducibility:** Use version control, requirements, and possibly Docker to ensure the project can be rerun.
- **Portfolio Quality:** Document everything, add tests, and produce a final demonstration video and report.

2. Target Users and Stakeholders

- **Primary Users:** Grid operators, utility analysts, energy researchers, or students learning ML for energy. The product is a demonstrator but should reflect real-world needs.
- **Stakeholders:**
- **Energy sector professionals:** They benefit from more efficient demand management.
- **City planners / sustainability advocates:** Forecasts aid renewable integration and demand response.

- **Project evaluators (instructor, thesis examiners):** Need a well-engineered system and clear documentation.
- **Developers (the student):** Gains portfolio material and learning.

No sensitive personal data is used, so privacy is not an issue. Data used will be aggregate load (no private identifiers).

3. Success Metrics and Acceptance Criteria

- **Forecast Accuracy:** We will report *Mean Absolute Error (MAE)* and *Root Mean Squared Error (RMSE)* on a held-out test set. As an acceptance criterion, the chosen model should reduce MAE/RMSE by at least **20% over a naive baseline** (e.g. persistence or "yesterday's load" forecast). Exact targets will be defined once data and baselines are computed.
- **Baseline Comparison:** A simple "naive" forecast (e.g. last hour's or last day's load) will be used as a baseline. The ML models should outperform this baseline.
- **Confidence Measures:** If possible, provide prediction intervals (e.g. via quantile regression) and evaluate using metrics like prediction interval coverage.
- **API Performance:** The prediction API should respond quickly. We set an acceptance threshold of **<500 ms** for generating a 24-hour forecast on a reasonable server.
- **Uptime/Reliability:** Aim for >99% availability of the web app. Continuous Integration tests will ensure that code changes do not break functionality.
- **Fairness/Risk:** Not applicable (no protected attributes). However, we will monitor model bias in terms of under/over-forecasting at peak times (e.g. never systematically underpredict peak load). A model that drastically underpredicts peak demand would be considered unacceptable.
- **User Acceptance:** The Streamlit UI should allow an end-user to select a date/time and view forecasts, with charts of *Actual vs Forecast*, error metrics, and an explanation (top features). A basic wireframe is described in the Appendix.

4. Data Sources and Licensing

Electricity Demand Data: We will use publicly available, hourly-level load data. Possible sources include:

- **NREL Historic Load Data (USA):** Estimated hourly demand for every US county (2016–2023) 【21†L100-L108】. This dataset (in HDF format) is open and licensed CC BY 4.0【27†L232-L240】, meaning it can be freely used with attribution. We can aggregate to a region or use as is.
- **International Data:** For variety, datasets like the European ENTSO-E Transparency Platform (aggregated load by country) or specific ISO/RTO data (e.g. PJM, CAISO) may be used if readily downloadable. Some Kaggle datasets exist (e.g. Romania hourly load), but their licenses may not allow redistribution, so caution is needed. We will prioritise CC-licensed or public-domain sources.
- **Weather Data:** Weather is a key feature. We will fetch historical temperature and other factors. Options include: NOAA's Global Historical Climatology Network (GHCN) hourly data (public domain) or a free API like *Open-Meteo* (no API key required) for forecast data. For example, including temperature and humidity from the relevant region is expected to improve accuracy 【33†L494-L501】.
- **Calendar/Holidays:** We will generate calendar features from known holiday calendars (country or region-specific). These are generally open data (e.g. major holidays).

- **Licensing:** All data will be used under their respective licenses. NREL and NOAA are open. If using any third-party API (e.g. OpenWeatherMap free tier), we will comply with terms of service. Since we do not collect personal data, privacy is not a concern.

Table – Example Dataset Sources:

Dataset	Scope	Source / License
NREL County-Level Load (2016–2023)	U.S. counties (all)	OpenEI/Data.gov (CC BY 4.0)【21†L100-L108】【27†L232-L240】
ENTSO-E or National ISO Load	EU countries	ENTSO-E Transparency (open)
PJM or CAISO historical load	U.S. ISO region	ISO public data portals
Romanian Hourly Load (Kaggle dataset)	Romania, 7.5 years	Kaggle (check license)
NOAA GHCN Daily/Hourly Temperatures	Global weather stations	NOAA/NCEI (public domain)

5. Forecast Horizon and Prediction Frequency

- **Horizon:** The project focuses on **short-term forecasting**, specifically *24 hours ahead* at **hourly resolution**. This means the model predicts the demand for each hour of the next day.
- **Prediction Frequency:** The system will be able to make rolling forecasts. For example, given data up to time *T*, it can predict *T+1 to T+24* hours. In practice, the user can request a forecast for the next 24 hours at any given hour (online forecasting). For evaluation, we will simulate this by cutting the historical series at the test point and forecasting the following day.
- **Rationale:** A 24-hour horizon is commonly used in grid operations and provides clear temporal structure (daily seasonality, nighttime vs daytime patterns). The assignment suggests "short-term," which typically means hours to days ahead. We will start with 24h and consider 48h as an optional extension if time permits.

6. Problem Formulation

- **Type:** This is a supervised *regression* problem. Inputs (features) include historical demand and exogenous data (time features, weather), and the target is the next-day hourly demand.
- **Data Splitting:** We must avoid "peeking into the future." The data will be split **chronologically** into: Training period (e.g. 2016–2020), Validation period (2021), and Test period (2022–2023). For model selection and tuning we use training+validation (e.g. last 20% hold-out within training), and the final unbiased performance is measured on the test split. No random shuffling is done; instead, we use time-series cross-validation for robustness.
- **Baseline Approaches:**
- **Persistence Model:** Forecast that demand at time *t+h* equals demand at time *t+h–m*, where *m* is a seasonal cycle (e.g. 24h or 168h). For example, "tomorrow's demand = today's demand at same hour" or "last week's demand".
- **Statistical Models:** ARIMA/SARIMA (autoregressive integrated moving average) or Prophet for comparison, which model seasonality and trend.

- **Evaluation:** Forecast accuracy will use MAE and RMSE, averaged over all predicted hours. We may also use *MAPE* or *SMAPE* carefully (watch zero-demand issues). We will report metrics on both validation and test sets.

7. Baseline and Candidate Models

A variety of models will be implemented and compared. Below is a preliminary list:

Model	Role / Notes
Persistence (Naive)	Baseline: use last observed value(s) (e.g. same hour yesterday/week) 【33†L494-L501】.
Seasonal Naive	Forecast by repeating the previous week's profile, adjusting for overall trend.
ARIMA / SARIMA	Classical time-series with automated seasonality (e.g. 24h, 168h weekly).
Prophet	Facebook/Meta Prophet: handles multiple seasonalities and holidays.
Linear Regression	Simple baseline with time and weather features (for interpretability).
Random Forest	Non-linear model capturing interactions, robust to outliers.
Gradient Boosting	XGBoost and/or LightGBM: powerful ensembles, often excel in tabular data.
LSTM (RNN)	Optional deep-learning for sequence data, if time permits.
Temporal CNN	Optional: 1D convolutional nets for time series.

Primary candidates will be Linear Regression, Random Forest, XGBoost, and possibly LightGBM, as these offer good performance with moderate effort. LSTM/TCN are listed as optional (they add complexity and training time). All models (except naive) will use the same input features for fairness.

> *We will select the final model based on empirical performance (lowest RMSE on validation) and simplicity. However, for a strong portfolio, we should train and show several models' results, then pick the best.*

8. Feature Engineering

Key features will include temporal, historical and exogenous information:

- **Time-based features:**
- Hour of day (0–23), day of week (Mon–Sun), month, whether it's a weekend/weekday, and major holiday flags.
- These capture obvious seasonal patterns (daily/weekly cycles) and special events (holidays often have lower demand).
- **Lag features:**
- Demand at *t-1*, *t-24* (same hour yesterday), *t-168* (same hour last week), etc. These give recent values as predictors.
- **Rolling statistics:**

- Rolling mean and standard deviation of past 24h, 48h, 168h windows (ending at time *t*). E.g., the average demand of the last day, last week, etc.
- **Weather features:**
- Temperature, humidity, and possibly wind or solar irradiance for the region. These are known to correlate with load (e.g. hotter days -> higher AC load). We will fetch historical temperature for the same timestamp. If reliable, hourly forecasts (or modeled forecasts) can even be used as input for future hours.
- **Calendar events:**
- Binary flags for public holidays (which can be found in libraries like `holidays` or government calendars).
- **Normalization:**
- For models like Linear Regression or Neural Nets, numerical features may be scaled (e.g. StandardScaler). Tree models do not require scaling.
- **Feature Selection:**
- Initially include all plausible features. After baseline runs, we may prune irrelevant ones or use regularisation to avoid overfitting.

These engineered features turn the time-series forecasting problem into a standard regression problem (with fixed-size input per prediction).

9. Data Preprocessing

- **Missing Data:** Check for any missing timestamps. If gaps exist, we will impute missing demand (e.g. linear interpolation or forward-fill). Weather data might have gaps; again interpolate or use nearest hourly.
- **Outliers:** We will inspect for anomalous spikes or drops (e.g. due to data issues). Outliers can be clipped or winsorized if they are obvious errors, but real peaks (e.g. heatwaves) should be kept as part of the signal.
- **Differencing (if needed):** For ARIMA we may difference the series to remove trend/seasonality. But for ML models, we use features instead.
- **Encoding:** Categorical variables (like day of week) will be one-hot encoded or cyclic-encoded (sine/cosine of hour if using a linear model).
- **Data Smoothing (optional):** To reduce noise, we might use a smoothing window. However, this risks blurring peaks. We will focus on raw data with robust models.
- **Quality Checks:** Ensure the preprocessing is deterministic (same operations on train/validation/test). We will pipeline preprocessing steps to avoid data leakage.

10. Model Training Pipeline

We will organize the code with a clear pipeline structure (examples in `/src/`). For instance:

1. **Data Loader:** Scripts to fetch/prepare raw data. If the source is an API or CSV, include code to download or update it.
2. **Preprocessing Function:** Applies the missing-value handling, feature engineering, and splits data into feature matrix `X` and target `y`.
3. **Training Script:** For each model, fit on training data, using validation (time-based CV). Use `scikit-learn`'s `TimeSeriesSplit` or similar for cross-validation, ensuring no leakage.
4. **Hyperparameter Tuning:** Use tools like `GridSearchCV` (with `cv=TimeSeriesSplit`) or a simpler loop over parameter grid. Given student constraints, keep parameter grids moderate.

5. **Model Persistence:** After training, save the best model (with its preprocessing pipeline) to disk (e.g. using `joblib`). This ensures the deployed API uses the identical pipeline as training.
6. **Evaluation:** After training, compute metrics on the validation and test sets. Compare MAE/RMSE to baseline. Possibly create a leaderboard or table of results.
7. **Calibration/Ensembles:** If models output probabilities or if we use quantile regression, calibrate using a validation set. Ensembles (averaging models) can be considered to improve robustness.

The pipeline should be reproducible end-to-end. Ideally, a single command or notebook cell runs data → train → evaluate.

11. Cross-Validation and Model Selection

- Use **time-series cross-validation**. For example, with `n_splits=5`, the folds might be:
- Fold 1: Train on months 1–12, validate on month 13
- Fold 2: Train on months 1–13, validate on month 14, etc. This respects chronology. We can implement this via `sklearn.model_selection.TimeSeriesSplit`.
- Metrics (MAE, RMSE) will be averaged across folds or reported per fold to check stability.
- The **final model** will be chosen based on lowest average validation RMSE (or MAE, depending on preference). We will also check that it's not overfitting (i.e. similar train/val errors).
- **Calibration:** After selecting a model, we may train it on the entire training+validation set and then evaluate on the test set for the final metrics.
- **Probabilistic Forecasts (optional):** If time permits, we might implement prediction intervals (e.g. using Quantile Regression Forest or by bootstrapping). This adds complexity, so it is lower priority.

12. Explainability

Even though this is a forecasting task, we will add interpretability:

- **Global Feature Importance:** For tree-based models, extract feature importances. We can present a bar chart of top features (e.g. "Hour of Day", "Temperature", "Lag24", etc.).
- **SHAP Values:** Use the SHAP library to compute local explanations. For a given forecast, SHAP can show how much each feature contributed to the prediction. This helps "explain" a specific forecast (e.g. "Peak temperature and hour 18 contributed to high demand").
- **PDP/ICE plots (optional):** Partial dependence plots for key features to show how forecasts change with temperature or hour-of-day.
- **Communication:** In the UI, include an "Explain" section where the user can click a forecasted time and see a SHAP force-plot or bar showing contributing factors. This transparency is valuable for trust.

13. Monitoring and Fairness/Risk

- **Monitoring:** In a real system, we would track performance over time (e.g. daily MAE) to detect drift. For this project, we will implement at least one check: e.g. after retraining, ensure model performance is consistent or improved.

- **Fairness:** This domain has no sensitive attributes, so fairness metrics (e.g. parity across groups) do not apply. However, **risk** of mis-forecasting is significant: underpredicting peak load can cause power shortages, while overpredicting wastes resources. We will mitigate this by cross-validating rigorously and possibly conservatively handling uncertainty (e.g. by alerting if predicted load is near historical peak).
- **Ethical Note:** We will state clearly in the documentation that this is a prototype for demonstration. Actual grid use requires much more validation and domain-specific safety checks. The system's forecasts are *predictions*, not guarantees.

14. System Architecture

A high-level architecture diagram:

```
 Historical Data (CSV/API) --> Data Pipeline --> Training Pipeline -->
Trained Model (artifact)
                                                      |
                                                      V
                                                Evaluation Reports

                New Data                          Prediction
           (last 24h / weather)                       |
                 |                                   V
             FastAPI Endpoint ---> Streamlit UI <--- Output forecasts, plots,
explanations
                 |
             Response (JSON)
```

- **Data Ingestion:** A script or notebook will pull in the raw data (e.g. CSVs of historical load, weather API calls). This can be a one-time manual fetch for historical data, or automated (cron) if using live feeds.
- **Storage:** We will likely keep data in local CSV/HDF files in the repo (or data folder). For a full product one might use a database, but that's not required here.
- **Training:** A Python module (`train.py`) reads the stored data, processes it, fits models, and outputs a saved model (e.g. `models/demand_model.pkl`) and metrics.
- **Model Registry:** For simplicity, the "registry" is just the versioned file in `models/`. If multiple models are tried, we can save them with versioned names.
- **FastAPI Backend:** A simple API (FastAPI) with at least one endpoint:
- `GET /predict?timestamp=YYYY-MM-DDTHH` which returns the next 24h forecast (or a JSON object with timestamps and predicted values).
- Possibly `POST /predict` with JSON input for batch queries.
- Maybe `/metrics` to return the latest model's validation metrics.
- **Streamlit Dashboard:** This will be the user interface. Key pages / sections could include:
- **Forecast Page:** User inputs a date/time and region (if multiple regions). The app shows a line chart of the next 24h forecast (and past actuals if available), and numeric values.
- **Model Comparison Page:** Displays performance (MAE/RMSE) of baseline vs chosen model, perhaps ROC or error distribution charts.
- **Explainability Page:** Shows SHAP explanations for a selected forecast hour or feature importance graph.

- **Data Overview Page:** (Optional) shows summary stats of the training data (mean load, seasonality plots).
- **Admin/Diagnostics:** (Optional) internal view of model configs or logs.
- **CI/CD:** We will use **GitHub Actions** to automatically run tests on each push or PR. Optionally, we can have an action to deploy the app (e.g. push to Streamlit sharing or Heroku) when merging to `main`. The CI could also rebuild models if data or code changes.
- **Environment:** A `requirements.txt` or `environment.yml` will list all Python dependencies. We may include a `Dockerfile` to containerize the app for deployment consistency (optional but strong for reproducibility).

Overall, the architecture is a single-repo "full-stack" ML project: data+code+app in one place, each layer (data, model, API, UI) connected but decoupled.

15. Testing Strategy

To ensure robustness, we will implement automated tests:

- **Unit Tests:** Using PyTest, we will test core functions. For example, tests for:
- Data processing: given a small synthetic dataset, verify feature engineering outputs the correct shape and values.
- Metric calculations: ensure MAE/RMSE functions give expected values on known inputs.
- API endpoints: using FastAPI's `TestClient` to check that `/predict` with a fixed timestamp returns properly structured JSON and reasonable values (we can use a dummy model or snapshot values).
- **Integration Tests:** A higher-level test might run the entire pipeline on a small subset: e.g. train on a toy series and check that the model file is created and that the Streamlit UI can load it without error. This is harder to automate but at least the build should import all modules without errors.
- **Regression Tests:** We can store example outputs (e.g. saved forecasts or metrics from a baseline run) and assert that new code does not dramatically worsen them. If metrics degrade, the CI can alert us.
- **Linting/Style:** (Optional) Add a linter (flake8) check in CI to enforce code quality.
- **Data Validation:** Check that incoming data (from source) matches expected format (e.g. correct number of columns, no NaNs after imputation).
- **Browser/Smoke Tests:** (Manual) Ensure the Streamlit app loads and displays expected elements. Automatic UI testing is out of scope.

Passing all tests will be part of the acceptance criteria (CI should show a green build before merging changes).

16. Deployment Options

Given the student budget (free or low-cost), we consider:

- **Streamlit Cloud (formerly Streamlit Sharing):** Free for public repos. It can host the Streamlit app directly. We can integrate the FastAPI by either:
- Running FastAPI within the same Streamlit script (with `st.experimental_http_request` or via threads).

- Or, deploying FastAPI separately (e.g. on Heroku or Railway) and letting Streamlit call it. The easier path is to have Streamlit call the model directly (skipping API) for the MVP. But since we want a production-like separation, we can deploy FastAPI on Heroku (free tier if still available) or Railway (free credits).
- **Heroku / Railway:** These platforms offer free or low-cost dynos. We can containerize the API (Docker) and deploy. Streamlit itself could be on Heroku as well, but Streamlit Cloud simplifies UI deployment. Note: Heroku discontinued truly free tier as of late 2023, but a credit card can give a small hobby tier. Railway has a $5/month free credit tier which may suffice.
- **GitHub Pages:** Not suitable (only for static sites).
- **GitHub Actions:** Could run workflows on push; can also deploy through actions (e.g. push Docker to registry, call cloud CLI to deploy).
- **Recommendations:** For minimal fuss, use Streamlit Cloud for the UI and bypass a separate API (Streamlit can import the model and return JSON via `st.json()` if needed). If an API endpoint is required by the assignment, deploy FastAPI on Railway (free up to a limit) or to a free App Engine (GCP free tier). We'll plan for Streamlit Cloud + Railway API.
- **Monitoring in Production:** Not needed beyond ensuring the app stays live. If usage is high, paid plans could be needed, but for a demo it's fine.

We will document deployment steps in the README (including any manual steps to start servers or set environment variables).

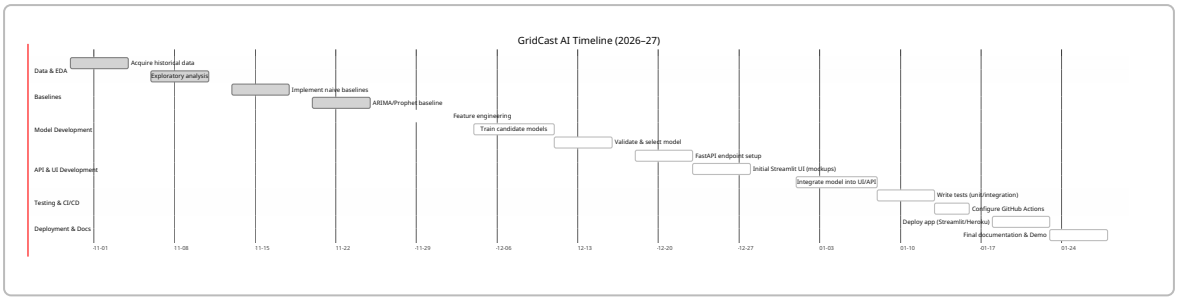
17. Reproducibility

To ensure others can reproduce our work:

- **Version Control:** All code will be in GitHub. Major pipeline steps will be script files (or notebooks saved). We will tag a release (e.g. v1.0) when complete.
- **Environment:** Provide a `requirements.txt` or `environment.yml` listing exact package versions. Alternatively, provide a `Dockerfile` with `python:3.x`, copying the code and installing dependencies. This eliminates “it works on my machine” issues.
- **Notebooks:** Include Jupyter notebooks for Exploratory Data Analysis and initial prototyping. These should be annotated so a reader can follow the thought process.
- **Data:** We will not upload large raw data to the repo. Instead, for the main dataset we choose (e.g. NREL HDF), we can either include a small sample or provide a download script with a short example. The README will document how to download the full data if needed.
- **Random Seeds:** For any randomness (e.g. train-test split, model training), set fixed seeds to make results reproducible.
- **Documentation:** The README will have instructions: setup environment, where to get data, how to train, how to run the app.
- **Demo Data:** To showcase the app, we might include a small CSV of “past 48h” data so the Streamlit app can load a quick demo without API calls.

18. Project Timeline (Gantt Chart)

The project is estimated at **8–10 weeks** of work. Below is a sample timeline using Mermaid Gantt notation (dates are illustrative, assuming a late-Oct 2026 start):



Each row represents tasks (see names for details). The chart shows sequential flow, but actual work may overlap (e.g. writing tests while finalizing UI).

19. Roles and Responsibilities

As a **student project**, the roles are self-assigned. For accountability, we can define:

- **Project Lead & Data Scientist:** (Your name) – Responsible for overall design, data acquisition, feature engineering, model development, and evaluation.
- **Software Engineer:** (Same or team) – Responsible for API and UI implementation, testing, and deployment infrastructure.
- **DevOps (CI/CD):** (Same) – Set up version control, GitHub Actions, and ensure reproducibility.
- **Quality Assurance:** (Same) – Write tests, review code, and document the system.

In practice, as a solo student project, you will perform all these tasks. However, listing them shows the full scope.

20. Budget and Tool Recommendations

Given the **student perspective and limited budget**:

- **Development Environment:** Use free tools. Python with IDE/Editor (VS Code or PyCharm Community). GitHub (free account) for code hosting.
- **AI Assistance:** ChatGPT (free tier) for research and planning. GitHub Copilot (Student pack is free) for coding help. No need to pay for multiple AI subscriptions initially.
- **Libraries:** All major libraries (scikit-learn, XGBoost, LightGBM, pandas, numpy, plotly, streamlit, fastapi, pytest, etc.) are open-source and free.
- **Hosting/Deployment:**
- *Streamlit Cloud:* free for public repos (may have usage limits but suitable for demo).
- *Heroku/Render/Railway:* free tier for API hosting (monitor usage). Alternatively, skip separate API and use Streamlit's server for predictions.
- **Weather API:**
- Use *Open-Meteo* (free, no key) or *NOAA* (free but has request limits).
- Free tiers suffice for development. A note: do not rely on a paid weather API (costly).
- **CI/CD:** GitHub Actions (free for open-source, 2000 free minutes).
- **Miscellaneous:** If needed, Google Colab can be used for heavy computation on a free GPU (for deep models), but our models should be fast enough on CPU. No paid GPU needed.

Overall: All required tools have free options. The only potential cost is if we want high-availability hosting, which is not needed for a student demo.

21. Risks and Mitigations

- **Data Unavailability:** If open data is sparse or patchy, the models may underperform. *Mitigation:* Collect multiple sources (e.g. use national and local data). Start with a small region if needed.
- **Model Underperformance:** It's possible our models may not beat the naive baseline. *Mitigation:* Ensure robust feature engineering and validation. Keep simpler models (LR) as back-ups. Focus on error analysis and iterate features.
- **Overfitting:** Risk of over-tuning to validation. *Mitigation:* Use strict time-series CV and test on held-out data. Keep model complexity in check.
- **Deployment Issues:** Free-tier platforms can be flaky or have downtime. *Mitigation:* Document deployment steps so alternative hosting is possible. For a demo, downtime is not critical.
- **Time Constraints:** Full implementation (multiple models, UI, tests) is a lot. *Mitigation:* Follow the timeline strictly, and prioritize a working end-to-end pipeline first (even with one model/UI page), then add features and polish.
- **API Limitations:** Free weather API might fail or throttle. *Mitigation:* Cache weather data or use dummy features if API fails.
- **Dependency Problems:** Library updates or environment issues may break code. *Mitigation:* Pin dependency versions and use virtual environments. Include a Dockerfile for stability.

Identifying these early helps avoid project stalls.

22. Appendix

A. Suggested Datasets and Links

- **Hourly Load Data:**
- NREL County-Level Load (2016–2023): [data.gov link】*(See Data Catalog ID: 3366592)*【21†L100-L108】 (License: CC BY 4.0【27†L232-L240】).
- ENTSO-E Transparency (EU country/regional load): https://transparency.entsoe.eu – free but requires registration.
- ISO/RTO open data (e.g. PJM, CAISO) – search for "[ISO name] historical demand data".

- Kaggle: *"hourly load consumption"* for various datasets (check individual licenses).

- **Weather Data:**

- *Open-Meteo API:* https://open-meteo.com – free, no API key. Offers temperature forecasts.
- *NOAA NCEI Data:* https://www.ncei.noaa.gov/ – free climate data (US weather stations).

- *Weather API (free-tier):* OpenWeatherMap free tier (requires API key), limited calls.

- **Holiday Calendars:**

- Python `holidays` library (pip install holidays) supports many countries, no extra license.

B. Example API Endpoints

- `GET /predict?region=XYZ&time=YYYY-MM-DDTHH:MM`
  Description: Return a JSON object with predicted demand for the next 24 hours starting at the

given timestamp (e.g. 2026-11-01T08:00).
Response: `{"timestamp": ["2026-11-01T09:00", ..., "2026-11-02T08:00"], "forecast": [1234.5, ..., 1100.2]}` (units in MW or relevant).

- `GET /metrics`
Description: (Optional) Return the latest validation/test metrics of the deployed model, e.g. MAE and RMSE on holdout data. Useful for monitoring.

- **Streamlit App Inputs:**

- A date/time picker for “Forecast start time”.
- A dropdown for region if multiple (e.g. county or country).
- A button “Run Forecast” that triggers the API call or local model inference.

C. Streamlit UI Wireframe (Textual)

1. **Header:** Title “GridCast AI: Demand Forecasting Dashboard”.
2. **Forecast Section:**
3. **Inputs:** Date-time selector (for forecast origin), Region selector.
4. **Outputs:**
    - *Line Chart:* Actual (if known) vs Forecast for the next 24h. X-axis: hour of day, Y-axis: demand.
    - *Forecast Table:* Tabular listing of hour and predicted demand.
    - *Error metrics:* If actual values for past are known, show error (for retrospective analysis).
5. **Model Comparison:**
6. Dropdown to select model (Baseline vs Final).
7. Plot of historical performance (e.g. scatter of predicted vs actual).
8. Metrics: MAE/RMSE in table.
9. **Explainability Panel:**
10. *Feature Importance:* Bar chart of top 10 global features.
11. *Local Explanation:* If the user clicks on a forecast hour, show a SHAP force-plot or bar for that prediction, highlighting major contributors (e.g. “High Temp (+20 MW), Peak Hour (+30 MW)”).
12. **About/Docs:** Short description of data sources, model methods, and contact/info.

(No images are embedded in this PRD, but the layout follows standard dashboard design.)
